An Attack on TON's ADNL Secure Channel Protocol

Aviv Frenkel
Fordefi
aviv@fordefi.com

Dmitry Kogan
Fordefi
dima@fordefi.com

***Abstract*—We present an attack on the Abstract Datagram Network Layer (ADNL) protocol used in The Open Network (TON), currently the 10th largest blockchain by market capitalization. In its TCP variant, ADNL secures communication between clients and specialized nodes called liteservers, which provide access to blockchain data. We identify two cryptographic design flaws in this protocol: a handshake that permits session-key replay and a non-standard integrity mechanism whose security critically depends on message confidentiality. We transform these vulnerabilities into an efficient plaintext-recovery attack by exploiting two ADNL communication patterns, allowing message reordering across replayed sessions. We then develop a plaintext model for this scenario and construct an efficient algorithm that recovers the keystream using a fraction of known plaintexts and a handful of replays. We implement our attack and show that an attacker intercepting the communication between a TON liteserver and a widely deployed ADNL client can recover the keystream used to encrypt server responses by performing eight connection replays to the server. This allows the decryption of sensitive data, such as account balances and user activity patterns. Additionally, the attacker can modify server responses to manipulate blockchain information displayed to the client, including account balances and asset prices.**

1. Introduction

The TON (The Open Network) is the 10th largest blockchain by market capitalization [1]. The TON ecosystem consists of multiple components, of which we focus on those most relevant to this work: validator nodes that process and verify transactions, liteservers that provide API access to blockchain data (including account balances, transaction history, and block data), and API providers that mediate access between liteservers and application clients, such as wallets and data indexers [2], [3]. The Abstract Datagram Network Layer (ADNL) serves as the primary secure transport protocol within the TON ecosystem. ADNL operates over two transports: the UDP variant [4], used for node-to-node communication in the peer-to-peer network, and the TCP variant [5], used for interactions with liteservers. A secure transport protocol must provide confidentiality, ensuring that transmitted data remains private from eavesdroppers, and integrity, guaranteeing that data cannot be modified in transit without detection. These security properties are crucial when transmitting sensitive financial and personal data. The canonical solution for achieving these properties is the Transport Layer Security (TLS) protocol [6], which has well-defined security goals, formal security proofs, and extensive real-world validation. However, TON's developers opted to design and implement ADNL as a custom secure channel protocol, departing from this established approach. Notably, ADNL lacks formal security definitions and proofs for its confidentiality and integrity properties. This is particularly concerning given the protocol's role in securing sensitive operations—including balance queries that reveal users' financial holdings and transaction history lookups that expose user activity patterns. The security of these operations directly depends on ADNL's ability to maintain confidentiality and integrity of the transmitted data.

In this work, we identify a cryptographic vulnerability in the TCP variant of the ADNL protocol. We then show how this vulnerability leads to a devastating attack on the protocol. Specifically, we show that a network adversary who intercepts an ADNL session between an honest ADNL client and an honest ADNL server can, by establishing a handful of additional sessions with the same server, decrypt the original intercepted session with high probability, thus breaking the protocol's *confidentiality*. Breaking the confidentiality of a session allows a network attacker to link a client's network identity and their blockchain wallet address. Moreover, an active network adversary can arbitrarily modify server responses while maintaining valid integrity checks, completely compromising the protocol's *integrity* property. Such an attacker can then forge server responses and spoof wallet balances and asset prices read by the client.

The vulnerability. The cryptographic design of the ADNL protocol contains some unfortunate choices, which include inadequate protection from replay attacks and a non-standard integrity-protection mechanism. The protocol consists of two phases: a handshake phase where parties establish shared secrets, and a data exchange phase where these secrets secure the communication. During handshake, the protocol employs semi-static Diffie-Hellman key exchange [7] with a fixed server public key, providing server authentication. However, the shared secrets are determined solely by client-generated values, with no fresh randomness provided by the server. This enables replay attacks, where

an adversary can reestablish identical session keys across multiple connections.

In the data exchange phase, the parties use the shared secrets established during handshake as keys for two AES-CTR stream ciphers, one for each direction of communication. Crucially, each cipher uses a key and an initialization value (IV) that remain *fixed* throughout the session. As a result, by replaying the client's handshake message, an attacker create different sessions with identical states of the stream cipher, and hence an identical key stream to be used for encrypting of the data of multiple sessions, leading to a classic two-time pad vulnerability. In this setting, an attacker who can predict portions of the plaintext in any of the sessions can potentially recover the keystream and decrypt the communications.

We then show how the compromise of ADNL's confidentiality leads also to a break of its integrity mechanism. Specifically, instead of using a keyed message authentication code (MAC), the ADNL protocol tries to protect the integrity of each message by hashing it with a random nonce, which is sent together with the message. As long as the protocol maintains confidentiality, and the nonce value used for the integrity check remains secret, an attacker that tampers with the message, does not know how to "fix" the hash value to pass the integrity check. This prevent naïve bit-flipping attacks on the protocol. However, an attacker that breaks the confidentiality of the protocol, learns the nonce used for the integrity check of each message. The attacker can then compute the correct hash value for any message of their choice. The attacker can then change the contents of each message and pass the integrity check by changing the hash value accordingly.

From a two-time pad vulnerability to a practical attack. Key reuse vulnerabilities in stream ciphers very often lead to practical breaks of the protocols [8], [9], [10], [11]. However, the specifics of the attacks, and their complexity depends on each particular protocol and the statistical properties of the plaintext data. For example, key-reuse attacks on natural-language plaintexts often use Markov models to recover the keystream [12]. Another classical approach is crib-dragging, a technique used at Bletchley Park during WWII [13], which exploits the presence of known or guessed plaintext segments to recover portions of the keystream.

While our research draws inspiration from these established techniques, the unique characteristics of our target system present a distinct algorithmic challenge. Our algorithm exploits two key patterns in ADNL communications. First, ADNL servers, known as liteservers, process multiple concurrent requests independently, causing response messages to be returned in varying order across different sessions. Our attack exploits this reordering property to obtain streams that contain identical plaintext segments at different positions in the keystream. Second, a substantial portion of ADNL traffic contains predictable data. ADNL API providers, used for example by wallets to read blockchain data, also run block indexers [3], which continuously collect newly mined blocks from the blockchain. The block data requests produce deterministic responses, providing the attacker with known plaintext segments.

To leverage these properties into an efficient attack, we first model the communication as streams that contains identical plaintext segments at different positions in each stream. Using this model, we construct an algorithm that efficiently recovers the encryption keystream by finding statistical correlations between predicted plaintext segments and XORed ciphertexts. We view our Keystream Recovery algorithm, developed specifically for this setting, as the main technical contribution of this work.

Implementation & Evaluation. We implement and evaluate a complete attack system that achieves passive keystream recovery and active message modification. Our implementation demonstrates the practical impact of this vulnerability by targeting a widely-used API provider. We provide a publicly available anonymized version of our implementation [14]. Our evaluation demonstrates the practical feasibility of this attack. Using 8 replay streams and only 6% known plaintext content, we achieve 80% keystream recovery. Through algorithmic optimizations, our implementation achieves keystream recovery throughput sufficient for real-time active attacks.

Summary of our contributions. In this work, we (i) identify a critical vulnerability in the ADNL protocol that compromises both the confidentiality and integrity of the protocol; (ii) construct a practical attack targeting an open-source ADNL client, which enables decryption of sensitive data and forgery of server responses; (iii) introduce a model and an algorithm for key-recovery attacks on protocols where predictable data appears at varying positions of the encrypted stream, which may be of independent interest.

Disclosure. Following responsible disclosure practices, we reported the vulnerability to the TON development team on November 18, 2024. The team acknowledged and addressed the issue. On November 26, 2024, they implemented a fix in the ADNL client component of their core library [15] that modifies the handshake to use a 3-phase protocol (see Section 8.1), This client-side fix remains compatible with existing server implementations, allowing for gradual adoption.

The TON team awarded us a bug bounty of $15,000. They confirmed their intention to eventually deprecate the insecure protocol flow on the server side, but will first ensure popular libraries have implemented the secure protocol. The TON team contacted major ADNL client implementations to advise on necessary changes.

The remainder of this paper is organized as follows. Section 2 describes the ADNL protocol and its security properties. In Section 3, we show the key-reuse vulnerability. Section 4 presents our attack model and the key properties we exploit. In Section 5, we detail our key-recovery attack, including the statistical plaintext model and a correlation-based algorithm. Section 6 demonstrates how the recovered keystream can be used to forge valid server responses. Section 7 presents our implementation

and evaluates its effectiveness. Section 8 discusses possible mitigations and explores a direction for future work.

1.1. Related Work

The ADNL protocol belongs to the class of secure channel protocols designed to provide confidentiality and integrity for communication over untrusted networks. Notable protocols in this class include Transport Layer Security (TLS) [6], Secure Shell (SSH) [16], and Internet Protocol Security (IPsec) [17]. Canetti et al. [18] established the theoretical foundations for analyzing such protocols by providing formal definitions of secure channels. An example of rigorous security analysis in this domain is the work of Jager et al. [19], who analyzed TLS by introducing the notion of authenticated and confidential channel establishment (ACCE).

Key-reuse vulnerabilities have appeared in various widely deployed protocols and applications. In a seminal work, Borisov et al. [8] demonstrated how the reuse of RC4 keystreams in the WEP protocol enabled practical attacks against 802.11 wireless networks. Schneier et al. [10] demonstrated how an attacker could force Microsoft's Point-to-Point Tunneling Protocol (PPTP) to reuse RC4 streams, creating a two-time pad vulnerability. Similarly, Wu [11] exposed how Microsoft Office applications reused RC4 keys across document revisions, allowing recovery of document contents. More recently, the KRACK attacks [9] showed how to force key reinstallation in WPA2, causing nonce reuse in various cryptographic algorithms, leading to packet decryption and forged packet injection. Böck et al. [20] demonstrated how nonce reuse in TLS implementations of AES-GCM enabled practical forgery attacks, highlighting the broader class of vulnerabilities that arise from reusing cryptographic state. A notable technique for exploiting key-reuse is crib-dragging, used by cryptanalysts at Bletchley Park during WWII - where known or guessed plaintext segments are used to recover portions of the keystream [13], a technique we also employ in our attack on ADNL.

Two-time pad attacks have a rich history, notably in the Venona project, where Soviet communication channels occasionally reused key material, enabling the decryption of diplomatic and intelligence messages [21]. More recent work by Mason et al. [12] developed a general technique for two-time pad attacks by modeling natural language plaintext as a Markov-chain and applying the Viterbi algorithm to solve a Hidden Markov Model (HMM) problem to recover the most likely plaintext. Our setting differs from Mason's work as we operate on network packets rather than natural language text. Rather than analyzing linguistic patterns, we exploit the presence of repeating patterns in ADNL communications, particularly in responses to replayed requests. This property allows us to construct a more specialized and effective plaintext model for our specific context. A recent work by Albrecht et al. [22] exploits AES-GCM IV reuse to recover plaintexts that differ by small edits. Their situation shares some similarities with ours in leveraging relationships between plaintexts. We discuss potential applications of their techniques to extend our attack in Section 8.2.

2. Background: The ADNL Protocol

In this section, we give some background about the TON blockchain and the role of the ADNL protocol within it. We then present the technical details of the ADNL protocol, focusing on its networking architecture and its cryptographic design.

2.1. The TON Blockchain

TON (The Open Network) is a fast-growing blockchain platform with a market capitalization exceeding $10B [1]. It employs a proof-of-stake consensus mechanism and leverages sharding techniques to achieve high scalability [23]. The platform supports a vast array of decentralized applications, from DeFi protocols [24] to gaming platforms [25]. The components of the TON blockchain that are relevant for this work are *network nodes*, *liteservers*, and *API providers*. These components communicate using the Abstract Datagram Network Layer (ADNL) protocol.

Network Nodes. TON nodes participate in the blockchain operation through peer-to-peer communication. The network includes full nodes that maintain the current blockchain state, archive nodes that maintain the complete blockchain history, and validator nodes that verify transactions and propose new blocks according to TON's Proof-of-Stake mechanism.

Liteservers. A liteserver is a functionality enabled on full nodes or archive nodes that allows client applications to interact with the TON blockchain. These servers process requests for account balances, transaction history, blockchain state, and enable transaction submission. TON's documentation maintains a list of global liteservers [26], where each server is identified by its IP address, port number, and Ed25519 public key.

The ADNL Protocol. The Abstract Datagram Network Layer (ADNL) provides transport security within the TON network. It operates in two modes: UDP for peer-to-peer communication between network nodes, and TCP for client-server communication with liteservers.

API Providers. API providers serve as intermediaries between applications and TON's liteserver network, offering HTTP-based interfaces [3]. Notable examples include TonAPI [27] and Toncenter [28]. These providers operate RPC nodes and indexers, enabling applications to query blockchain data without direct ADNL connections to liteservers.

Clients interact with the TON blockchain either through direct communication with liteservers using ADNL, or through HTTP requests to API providers. TON's documentation maintains a curated list of client SDKs [29] implementing both approaches across multiple programming languages.

2.2. The ADNL Protocol

The Abstract Datagram Network Layer (ADNL) provides transport security within the TON ecosystem. While ADNL operates over both UDP for node-to-node communication and TCP for interactions with liteservers, this paper examines the TCP variant of the protocol. In this section, we describe the protocol in detail, analyze its security requirements, and present a vulnerability that compromises both the confidentiality and integrity of communications between clients and liteservers.

To establish a connection with a liteserver, a client first obtains the server's IP address, port number, and Ed25519 public key, for example from the global TON configuration [26]. The client then establishes a TCP connection to the server. The protocol consists of two phases. The handshake phase takes the server's public key as input and involves sending a single packet from the client to the server. Upon a successful handshake, both parties establish shared symmetric encryption keys. Following the handshake, the client and server exchange encrypted datagram messages using these established keys.

2.2.1. Handshake. Starting with the server's Ed25519 public key, the ADNL handshake requires only a single message from the client to establish the session's cryptographic material. The client first generates an ephemeral, 256-bit, Ed25519 key pair and performs a X25519 key exchange [30] with the server's public key, producing a shared secret. The client then generates symmetric session parameters (`aes_params`) containing encryption keys for the subsequent communication. These parameters are encrypted using keys derived from the shared secret, and sent to the server along with the client's public key and a hash for verification. Upon handshake completion, both parties initialize two AES-CTR [31] stream ciphers using `aes_params`. These stream ciphers generate keystreams $k_{c\to s}$ and $k_{s\to c}$ for client-to-server and server-to-client communications respectively. More concretely, in AES-CTR mode, the 16-byte nonce from `aes_params` is interpreted as a 128-bit counter value. To generate the keystream, the counter value is encrypted with AES using the session key (either `rx_key` or `tx_key`), producing a 16-byte keystream block. After each keystream block generation, the counter value is incremented by one (modulo 2^{128}) for the next block.

2.2.2. ADNL Query. Following the handshake phase, communication with liteservers follows a request-response pattern. Each request is assigned a randomly generated 32-byte query identifier (`query_id`). When responding to a request, the liteserver includes the corresponding `query_id` in its response, enabling the client to match responses with their originating requests. This query identification mechanism allows responses to be delivered in any order.

2.2.3. Packet Encryption and Integrity. When a client wants to send a request to the server, it first constructs a datagram according to the structure shown in Table 1. For a given payload buffer, the datagram includes a randomly generated 32-byte nonce and a hash value computed as SHA-256(`nonce`‖`buffer`). The client then encrypts this datagram by XORing it with the next segment of keystream $k_{c\to s}$. Similarly, the server constructs its responses and encrypts them using keystream $k_{s\to c}$. Each encryption advances the stream cipher's internal state, progressing the keystream for subsequent datagrams.

ConstructHandshakePacket(P_{Server}):

Input: Server's public `Ed25519` key P_{Server}

1: Generate fresh `Ed25519` key-pair: S_{Client}, P_{Client}

2: Compute shared secret:
`secret` ← X25519(S_{Client}, P_{Server})

3: Generate session parameters:
`rx_key` ← random(32 bytes)
`tx_key` ← random(32 bytes)
`rx_nonce` ← random(16 bytes)
`tx_nonce` ← random(16 bytes)
`padding` ← random(64 bytes)

4: `aes_params` ← `rx_key` ‖ `tx_key` ‖ `rx_nonce` ‖ `tx_nonce` ‖ `padding`

5: Encrypt session parameters:
`hash` ← SHA256(`aes_params`)
`key` ← `secret`[0..16] ‖ `hash`[16..32]
`nonce` ← `hash`[0..4] ‖ `secret`[20..32]
E ← AES256_CTR(`key`, `nonce`)

6: **return** P_{Client} ‖ `hash` ‖ E(`aes_params`)

Figure 1. ADNL (TCP) - Handshake packet construction

TABLE 1. ADNL (TCP) - DATAGRAM STRUCTURE

Parameter	Size	Notes
length	4 bytes (LE)	Length of the whole datagram, excluding length field
nonce	32 bytes	Random value
buffer	`length` − 64 bytes	Data to be sent
hash	32 bytes	SHA-256(`nonce`‖`buffer`)

The SHA-256 hash field in ADNL's datagram serves as its integrity mechanism [23], following a pattern similar to MAC-then-encrypt, where a MAC is computed over the plaintext before encryption. However, ADNL's approach differs from standard MAC constructions. Rather than using a keyed hash function, it relies on the confidentiality of the nonce to prevent tampering - the hash SHA-256(`nonce`‖`buffer`) provides no cryptographic integrity without this assumption. This design choice contrasts with the more widely accepted encrypt-then-MAC pattern, where the MAC is computed over the ciphertext. While MAC-then-encrypt can be implemented securely when properly constructed [32], the current ADNL implementation

lacks the formal security properties typically expected from a MAC function.

3. Key Reuse Vulnerability

The TCP variant of ADNL protocol, as defined in Section 2.2, is susceptible to a replay attack. The handshake requires no input from the server in establishing session encryption parameters (`aes_params`), placing the AES-CTR key derivation entirely under client control. During handshake, the client generates both the symmetric keys and initialization vectors used by both parties, allowing an attacker to replay a captured handshake packet to establish a session with identical encryption parameters to the victim's original session. When replaying the client's encrypted datagrams in this new session, the server successfully decrypts and processes them as legitimate requests, responding with fresh encrypted datagrams using the same encryption parameters as the original session. This process can be repeated multiple times, establishing new sessions that share the same encryption parameters. Since these parameters determine the AES-CTR keystream, replayed sessions produce identical keystreams for both directions of communication $k_{c\to s}$ and $k_{s\to c}$. When two ciphertexts c_i and c_j from different sessions are encrypted using the same keystream k, their XOR eliminates the keystream: $c_i \oplus c_j = (p_i \oplus k) \oplus (p_j \oplus k) = p_i \oplus p_j$, revealing the XOR of their underlying plaintexts. This reduction to a two-time pad creates a critical weakness that forms the foundation of our attack.

ADNL's integrity protection mechanism deviates from standard practice by relying on confidentiality rather than using a proper message authentication code (MAC). For each datagram, the protocol generates a random 32-byte nonce and computes $\text{SHA-256}(\texttt{nonce}\|\texttt{buffer})$ as an integrity check. When an attacker recovers the keystream through the two-time pad attack, they can decrypt the nonce from captured messages, modify the message content arbitrarily, compute new valid integrity hashes using the decrypted nonce, and re-encrypt the modified message and hash using the known keystream.

In the remaining sections, we demonstrate how to exploit this two-time pad vulnerability to reconstruct the AES-CTR keystream. The recovered keystream enables both passive decryption of intercepted communications and active message manipulation by computing valid integrity hashes with the decrypted nonces.

4. The Attack Model

In Section 3, we have identified a critical vulnerability in the ADNL protocol that compromises its confidentiality and integrity guarantees. While this vulnerability affects the protocol's design in general, we demonstrate its practical exploitation through a concrete implementation targeting specific deployments. This section outlines our attack environment, analyzes the properties that enable successful exploitation, and discusses the broader implications and limitations of our approach.

4.1. Attacker Capabilities

Our attack model considers a man-in-the-middle network adversary positioned between the target client and the liteservers it connects to. This adversary has complete visibility of all ADNL traffic flowing between these endpoints and can modify transport layer communications, including the ability to inject TCP packets and terminate TCP sessions. Additionally, the adversary can establish independent connections to the same liteservers that the target client communicates with. In real-world scenarios, such an adversarial position could be achieved through compromise of network infrastructure such as routers, switches, or proxies along the communication path between API providers and liteservers. For example, an attacker might exploit vulnerabilities in the Border Gateway Protocol to reroute traffic destined for liteservers through attacker-controlled infrastructure [33].

4.2. Target Environment

Our attack implementation targets *opentonapi* [34], an open-source API provider that serves as a gateway for various TON applications, including the widely-used Tonkeeper wallet [35]. The attack succeeds despite opentonapi following standard practices and recommended implementation patterns. Since the vulnerability lies in the ADNL protocol design rather than implementation-specific issues, other API providers and ADNL clients may also be vulnerable to this attack. The implementation operates against production liteservers (as of October 2024), demonstrating the vulnerability's relevance to current deployments. We selected this environment because API providers represent a significant portion of ADNL traffic, making them a particularly relevant target for security analysis. Several factors contribute to a deployment's susceptibility to exploitation:

- Network separation between client and liteserver
- High-volume ADNL connections carrying mixed traffic types
- Predictable patterns in request/response data
- Reliance on ADNL's transport security without additional application-layer protections

4.3. Properties of the Target Environment

While the fundamental vulnerability in ADNL's design can potentially be exploited in various contexts, our attack implementation leverages specific properties of our target environment to demonstrate a practical attack. The following properties, while not necessary requirements for exploiting the core vulnerability, allow us to construct an efficient and reliable attack in our chosen setting:

Request Multiplexing in Liteservers. In our analysis of TON's global liteservers, we observed a property we term *Request Multiplexing*. When multiple ADNL queries are

sent over the same connection, their corresponding responses may arrive in an order different from the original request sequence. Moreover, this ordering is not constant - replaying identical request sequences to the same liteserver can yield responses in varying orders. This behavior results from concurrent query handling in liteservers. When multiple requests arrive together, they are processed concurrently, and their responses are multiplexed over the shared ADNL connection. As we demonstrate in Section 7.2.1, simultaneous arrival of requests increases the likelihood of response reordering. This property plays a crucial role in our attack methodology.

Opentonapi - Block Indexer. In the default configuration of opentonapi, the service operates a block indexer. A block indexer is a process that continuously queries liteservers to collect and store block data. The indexer performs these queries using `getBlock` requests to liteservers. Each `getBlock` request specifies the block's timestamp as a parameter and receives the complete block content as a response. Similar block collection functionality is likely present in other API provider implementations, as it enables basic blockchain data access.

Opentonapi - Connection Management. The opentonapi service establishes ADNL connections to multiple liteservers upon startup. In the default configuration, it connects to all available global liteservers. Among these connections, the service designates a "best connection" based on ping time measurements. This connection receives priority and carries the majority of the traffic. Multiple application components share these ADNL connections. For instance, the block indexer's `getBlock` queries and wallet-related `getBalance` queries can traverse the same ADNL connection.

4.4. Limitations

Our attack demonstration has several important limitations that warrant discussion:

Protocol Scope. The identified vulnerability specifically affects the TCP variant of ADNL. The UDP variant, which serves different use cases within the TON ecosystem, is not affected by this vulnerability.

Deployment Considerations. Our attack model assumes network separation between the client and the liteserver. While this represents a viable deployment scenario, some implementations may co-locate these components or employ private network configurations that prevent network-based attacks. However, the underlying cryptographic vulnerability remains relevant for any deployment that relies on ADNL over TCP for secure communication.

Application-Layer Mitigations. The protocol's security limitations can be partially mitigated through application-layer measures. For instance, responses could be validated against the blockchain state using Merkle proofs [36]. However, such validation introduces performance overhead and is not universally implemented. Many applications, including our target environment, opt to trust the data provider directly, prioritizing performance over additional security guarantees.

4.5. Generalizability

Among the properties outlined in Section 4.3, our attack fundamentally relies on two assumptions: server-side request multiplexing and predictable client traffic. While the server's multiplexing behavior appears to be a robust assumption, the assumption of predictable client traffic, represents a stronger requirement. In Section 8.2, we present a theoretical extension of our attack that could eliminate the need for plaintext prediction.

5. Breaking ADNL's Confidentiality

This section outlines the framework of our attack against ADNL's confidentiality. At its core, our attack exploits the key-reuse vulnerability to mount a two-time pad attack. When two ciphertexts c_1, c_2 encrypted with the same keystream k are XORed together, the keystream cancels out: $c_1 \oplus c_2 = (p_1 \oplus k) \oplus (p_2 \oplus k) = p_1 \oplus p_2$, leaving only the XOR of the underlying plaintexts. The success of such an attack relies on leveraging information about the plaintexts to recover them from their XOR.

Converting this relation into a practical decryption attack presents three technical challenges. First, we need a reliable way to obtain multiple messages encrypted with the same keystream. Second, we need a model that captures the non-uniform structure of ADNL messages. Third, we need an efficient algorithm to recover plaintexts using this model.

This section develops a decryption attack by addressing each of these challenges. In Section 5.1, we establish our data collection methodology, showing how to obtain multiple ciphertexts through connection replay and gather predicted plaintexts by exploiting the deterministic nature of certain ADNL responses. In Section 5.2, we present our plaintext model and develop an efficient keystream recovery algorithm that exploits the model's properties. The performance of our this attack - specifically its ability to decrypt messages in real-time using minimal replay sessions and predicted plaintext - is crucial as we will later show how it enables an active attack that compromises message integrity. In Section 5.3, we discuss the implications of breaking ADNL's confidentiality, particularly how it enables deanonymization attacks on TON users.

5.1. Data Collection

Our attack requires gathering multiple ciphertexts encrypted with the same keystream and obtaining information about their underlying plaintexts. To this end, we collect three types of data. First, we intercept an ADNL session between a client and a server. Then, exploiting ADNL's key reuse vulnerability, we replay the client's requests to obtain additional response streams encrypted with the same keystream. Finally, we gather plaintext segments that we expect to find within these response streams. The combination

of replayed streams and predicted plaintexts provides the foundation for our keystream recovery algorithm.

Intercepted Communications. We consider an ADNL session containing n requests. We denote:

- s: The client-to-server ADNL stream, including the handshake packet and subsequent encrypted datagrams.
- p: The server-to-client plaintext stream.
- c: The server-to-client ciphertext stream.
- k: The server-to-client keystream, where $c = p \oplus k$.

Replayed Communications. By exploiting the key reuse vulnerability described in Section 3, we replay the client's stream s to obtain N response streams. For each replay index $1 \leq i \leq N$:

- p_i: The i-th server-to-client plaintext stream.
- c_i: The i-th server-to-client ciphertext stream, where it holds that $c_i = p_i \oplus k$.
- $\sigma_i : [n] \to [n]$: A permutation over $\{1, \ldots, n\}$ representing the reordering of response packets relative to their corresponding requests.

Each plaintext stream p_i consists of n packets: $p_i = p_i^1 \| ... \| p_i^n$ where $p_i^{\sigma_i(j)}$ is the response to the j-th request. These σ_i permutations arise from the Request Multiplexing property of liteservers discussed in Section 4.3. The relative positioning of identical packets across different streams is crucial for our attack. When a packet appears at the same position in two streams, its contribution to the XORed ciphertexts cancels out completely, yielding no information about the underlying keystream. However, when the same packet appears at different positions, each time against different plaintexts in the other streams, it provides useful information about the underlying keystream. To quantify the effectiveness of this reordering for our attack, we define the derangement ratio: $D(\sigma) = \frac{|j:\sigma(j)\neq j|}{n}$. This ratio measures the fraction of responses appearing at positions different from their corresponding requests. This measure serves as a proxy for the actual quantity of interest: the number of different positions at which the same packet appears across streams. In the worst case, when there is no reordering ($D(\sigma_i) = 0$), each packet appears at the same position across all streams, providing no useful information for our attack. As $D(\sigma_i)$ increases, it improves our ability to recover the keystream. Section 7.2.1 shows that we can increase the value of $D(\sigma)$ by transmitting requests in rapid succession. In Section 7.2.3 we evaluate the effect different values of $D(\sigma)$ has on the performance of our attack.

5.1.1. Predicted Plaintexts. Let $\hat{P} = \{\hat{p}_1, ..., \hat{p}_M\}$ denote M predicted plaintext segments. We define $\mu : \hat{P} \times \{p_1, ..., p_N\} \to \{0, 1\}$ where $\mu(\hat{p}, p) = 1$ if $\hat{p}$ appears as an exact substring within the plaintext stream p, and 0 otherwise. Let $L = \frac{\sum_{\hat{p}\in\hat{P}} \mu(\hat{p},p)\cdot|\hat{p}|}{|p|}$ denote the fraction of the plaintext stream that is covered by segments from $\hat{P}$. In the setting of our attack, where the client is assumed to run a block indexer, we have access to such plaintexts through `getBlock` requests. We can construct `getBlock` queries that match those we expect to find in the underlying traffic. In Section 7.1, we detail our strategy for constructing these matching queries based on request timestamps. In Section 7.2.3 we show how different values of L affect the success rates of the keystream recovery algorithm.

5.2. Keystream Recovery

Given our collected data—multiple ciphertext streams $\{c_1, ..., c_N\}$ encrypted with the same keystream k, and a set of predicted plaintext segments $\hat{P}$—we now develop an algorithm for recovering segments of k. We first define a plaintext model that captures the properties outlined in the previous section, then develop the Segment Matching algorithm which serves as the core primitive of our algorithm. We then describe the complete key-recovery algorithm which uses Segment-Matching iteratively to recover increasingly larger patches of the keystream.

5.2.1. Plaintext Model. Two-time pad attacks rely on plaintexts being sampled from non-uniform distributions. A plaintext model characterizes this non-uniformity by defining properties that distinguish valid plaintexts from random bitstrings. For the ciphertext streams $c_1, ..., c_N$ collected using the approach detailed in Section 5.1, and their corresponding underlying plaintexts $p_1, ..., p_N$, we define two distinguishing properties:

- Predicted Segment Occurrence: For each plaintext stream p_i, segments from our prediction set $\hat{P}$ appear as substrings with non-negligible probability.
- Cross-Stream Repetitions: For each stream p_i, significant portions of its content appear as substrings in other streams p_j. Due to the message reordering described in Section 4.3, some of these repeated sequences occur at different offsets within each stream.

In developing our plaintext model, we considered common approaches in cryptanalysis for modeling non-uniform distributions of plaintexts. Markov chains provide a powerful framework for capturing statistical properties in structured data: they model sequences where each element's probability distribution depends on a finite history of previous elements. Previous works have exploited this model in various structural languages, including natural languages, where the distribution of words, sentences, and larger linguistic structures can be modeled using Markov chains, reducing the recovery problem to solving a Hidden Markov Model [12].

However, the specific characteristics of ADNL traffic differ significantly from natural language texts. In our setting, rather than knowing that the plaintext follows some statistical distribution, the attacker has access to some set of known plaintext segments. While we could model this using a Markov chain approach, solving the resulting Hidden Markov Model using the Viterbi algorithm would yield complexity $O(l^2)$, where l represents the length of the

ciphertext and predicted plaintext (which are comparable in size). This complexity is inefficient for our real-time attack requirements. Instead, by directly leveraging the predictable message patterns and repeated segments in ADNL traffic, we can extract information more efficiently than through generic statistical modeling. Our plaintext model is therefore built around repetitions and long predicted segments, following a crib-dragging approach [13]. This tailored approach not only better captures the specific properties of ADNL communications but also enables a more efficient recovery algorithm with complexity $O(l \log l)$ instead of $O(l^2)$. This efficiency gain is crucial for making our attack practical in real-world scenarios where real-time performance is essential.

5.2.2. Segment Matching. At the core of our keystream recovery algorithm lies the ability to identify plaintext segments within encrypted streams. Consider two cipher streams c_1, c_2 encrypted using the same keystream k, and a candidate plaintext segment $\hat{p}$ with length $|\hat{p}| = l$. Let $\delta = c_1 \oplus c_2$, from the properties of stream ciphers, we know that $\delta = p_1 \oplus p_2$, eliminating the keystream k from our equations. When $\hat{p}$ appears as a segment in p_1 at offset i, we have $p_1[i : i+l] = \hat{p}$, and consequently $p_2[i : i+l] = \delta[i : i+l] \oplus \hat{p}$, same goes when replacing the roles of p_1 and p_2. For every other offset, the result of $\delta[i : i + l] \oplus \hat{p}$ is a XOR of three plaintexts, specifically $p_1[i : i + l] \oplus p_2[i : i + l] \oplus \hat{p}$.

The statistical difference between these two cases—a single plaintext versus XOR of three plaintexts—becomes detectable and significant when working with sufficiently long plaintexts. To identify this statistical difference we use the monobit frequency test [37]. The test examines whether a sequence's bit distribution matches that of a truly random sequence. For a bitstring B of length n, we define the test statistic $S_n(B) = \sum_{i=1}^{n} (-1)^{B_i}$. For random bitstrings, S_n follows a half-normal distribution. Testing $\delta[i : i + l] \oplus \hat{p}$ at each offset i yields two distinct scenarios: When i corresponds to a true match, we observe the distribution of a single plaintext. For all other offsets, we observe the distribution of a XOR of three plaintexts.

The XOR of three plaintexts produces a distribution much closer to random than that of a single plaintext. This distinction provides a discriminator: when testing all possible offsets, most positions yield S_n values close to zero, while matching offsets produce values that significantly deviate from zero. By computing the corresponding P-values under the half-normal distribution, we can identify potential matches as those positions where the observed frequencies show the strongest deviation from randomness.

The test described above is statistical in nature and may therefore introduce false positives. For instance, when packets contain varying fields, partial matches may pass our statistical criteria. Since any false positive in this segment matching may propagate errors throughout our key-extraction algorithm, we require exact matches. We maintain this exactness through multiple validation steps: matches must demonstrate overwhelming statistical significance (achieved by tuning parameters T and α in Algorithm 1), be consistent across additional ciphertext pairs, and align with known plaintext patterns. Furthermore, we verify the SHA-256 integrity hash of any extracted plaintext datagram (obtained by XORing out the candidate segment), providing cryptographic assurance of the match quality beyond statistical measures.

Algorithm 1 Segment Matching

Require: XORed ciphertexts $\delta = c_1 \oplus c_2$, candidate plaintext $\hat{p}$ of length l where $l \leq |\delta|$, significance parameters α, T {typically $T < 10$},
1: **for** each valid offset i **do**
2: $B \leftarrow \delta[i : i + l] \oplus \hat{p}$
3: Compute $S_n(B)$ and the corresponding P-value
4: **end for**
5: Let p_{T+1} be the $(T + 1)$-th most extreme P-value
6: // Use a threshold to filter in only significant offsets
7: **return** $\{i : \text{P-value}_i < \alpha \cdot p_{T+1}\}$

5.2.3. Segment Matching - Efficient Computation. The segment-matching test described in Algorithm 1 requires computing S_n values for every possible offset. A direct implementation would involve a large loop iterating through all offsets, making it prohibitively slow for practical use. We can improve the performance by reformulating the computation in terms of cross-correlation, which can be efficiently computed using Fast Fourier Transform (FFT) [38], a well-established technique in signal processing literature for optimizing correlation computations [39].

Specifically, by transforming our bitstrings into sequences of $\{-1, 1\}$, computing S_n at offset i becomes equivalent to computing the cross-correlation at that offset. The FFT-based implementation allows us to compute these values efficiently for all possible offsets, reducing our time complexity from $O(|\delta| \cdot l)$ to $O(|\delta| \log |\delta|)$ for each candidate plaintext.

In our key recovery algorithm, we process multiple ciphertext pairs against multiple plaintext candidates. Let $\Delta = \{\delta^i \oplus \delta^j\}$ be our set of XORed ciphertext pairs and $\hat{P} = \{\hat{p}^k\}$ be our set of plaintext candidates. Computing cross-correlation for every pair in $\Delta \times \hat{P}$ requires $|\Delta| \cdot |\hat{P}|$ cross-correlation operations. Each cross-correlation requires three FFT computations, following the convolution formula: $x * y = FFT^{-1}(FFT(x) \cdot FFT(y))$ [38]. We observe that the convolution operands x, y are independently derived from $\delta \in \Delta$ and $\hat{p} \in \hat{P}$. By precomputing $FFT(x)$ once for each $\delta \in \Delta$ and $FFT(y)$ once for each $\hat{p} \in \hat{P}$, we reduce the total number of FFT operations from $3 \cdot |\Delta| \cdot |\hat{P}|$ to $|\Delta| + |\hat{P}| + |\Delta| \cdot |\hat{P}|$.

We further optimize this computation by exploiting the underlying structure of ADNL messages. The naive FFT-based approach computes cross-correlation for all possible bit-level offsets, introducing unnecessary computation. A property of ADNL messages enables further optimization: the TL (Type Language) [40] serialization used in ADNL queries ensures all datagrams are 32-bit aligned, meaning any offset differences between δ and $\hat{p}$ must be a multiple

of 32 bits. Instead of computing all $(n - l + 1)$ bit-level offsets, we can split the streams into 32 binary channels and compute cross-correlation separately for each channel. This reduces our problem to computing 32 cross-correlations on operands 32 times smaller, yielding only the meaningful $(n/32 - l/32 + 1)$ offsets. While this transformation maintains the same asymptotic complexity, it offers some memory reduction and a clean tradeoff between computational cost and detection accuracy, preferable to alternative tradeoffs such as truncating input streams.

We incorporate these optimizations into our *Batched Segment Matching* algorithm (Algorithm 2), and evaluate their effectiveness in Section 7.2.2.

Algorithm 2 Batched Segment Matching

Require: Set of XORed ciphertext pairs Δ, set of plaintext candidates $\hat{P}$, number of channels $k \leq 32$

// Precompute FFTs
for each $\delta \in \Delta$ **do**
  Split δ into k binary channels: $\delta_1, ..., \delta_k$, where $\delta_i[j] = \delta[32j + i]$
  $F_\delta \leftarrow FFT(\delta_1), ..., FFT(\delta_k)$
end for
for each $\hat{p} \in \hat{P}$ **do**
  Split $\hat{p}$ into k binary channels: $\hat{p}_1, ..., \hat{p}_k$
  $F_p \leftarrow FFT(\hat{p}_1), ..., FFT(\hat{p}_k)$
end for
// Compute cross-correlations
for each $(\delta, \hat{p}) \in \Delta \times \hat{P}$ **do**
  for channel $i \in 1, ..., k$ **do**
    $corr_i \leftarrow FFT^{-1}(F_\delta[i] \cdot F_p[i])$
  end for
  Sum channel correlations to compute S_n values
  Apply threshold to identify matching offsets
end for
return All identified matching offsets

5.2.4. Keystream-Recovery Algorithm. Building on the batched segment matching algorithm, Algorithm 3 implements an iterative process for keystream recovery. The algorithm maintains two states: a set of recovered keystream patches K and a set of predicted plaintexts P_{pred}, initially populated with our predicted plaintext segments $\hat{P}$. The algorithm starts by computing N XORed ciphertext streams as $\Delta_i = c_i \oplus c_{i+1}$ (cyclically). The algorithm then iteratively invokes Algorithm 2. In each iteration, when Algorithm 2 identifies a matching segment $\hat{p}$ in Δ_i at offset j, it indicates that our candidate packet $\hat{p}$ appears in either c_i or c_{i+1} at that offset. By checking if offset j appears as a match in both Δ_i and Δ_{i-1}, we can determine which of the two ciphertext streams contains $\hat{p}$. Once identified, we recover the corresponding keystream segment by XORing the plaintext with its matching ciphertext. Each recovered keystream segment enables the decryption of the corresponding segments in that position in all other ciphertext streams. Due to the server's response multiplexing behavior described in Section 4.3, these newly decrypted segments overlap with different response packets across streams, potentially revealing additional plaintext data. The algorithm maintains two types of recovered plaintext segments as candidates for the next iteration of batched segment matching:

1. Complete packets: Plaintexts for which we can verify the ADNL integrity check by computing their SHA-256 hash. This cryptographic validation helps eliminate false positives that may arise from the statistical nature of our segment-matching test.

2. Repeated segments: Sequences that were recovered multiple times across different streams. These segments often represent deterministic response data and are therefore likely to appear in other replayed streams as well, making them valuable candidates for subsequent matching iterations.

Algorithm 3 Keystream Recovery

Require: Ciphertext streams $C = \{c_1, ..., c_N\}$, initial predicted plaintexts $\hat{P}$

$K \leftarrow \emptyset$ // Set of recovered keystream patches
$P_{pred} \leftarrow \hat{P}$ // Current set of predicted plaintexts
// Compute XOR pairs
$\Delta \leftarrow c_i \oplus c_{(i \bmod N)+1} : 1 \leq i \leq N$
while $P_{pred} \neq \emptyset$ **do**
  // Batch segment matching
  $M \leftarrow$ BatchSegmentMatch(Δ, P_{pred}) // Returns matches $(i, j, \hat{p})$ where i is the index in Δ, j is the offset, and $\hat{p}$ is the matched plaintext
  $P_{new} \leftarrow \emptyset$ // New plaintexts discovered in this iteration
  for each match $(i, j, \hat{p}) \in M$ **do**
    if offset j appears in both Δ_i and Δ_{i-1} **then**
      // Recover keystream segment
      $k_{new} \leftarrow c_i[j : j + |\hat{p}|] \oplus \hat{p}$
      Add (j, k_{new}) to K
      // Decrypt new plaintexts using recovered keystream
      for $l \in \{1, ..., N\} \setminus \{i\}$ **do**
        $p_{new} \leftarrow c_l[j : j + |\hat{p}|] \oplus k_{new}$
        $packet \leftarrow$ ExtractPacket(p_{new})
        if ValidatePacketHash$(packet)$ and $packet \notin P_{pred}$ **then**
          Add $packet$ to P_{new}
        end if
      end for
    end if
  end for
  $P_{pred} \leftarrow P_{new}$
end while
return K

5.3. Attack Implications: Deanonymization Attack

Breaking ADNL's confidentiality has significant privacy implications for TON users. When users interact with the TON blockchain through wallet applications, they typically

connect to API providers that act as intermediaries, which in turn use ADNL to communicate with liteservers. In this scenario, we assume a network adversary who can observe both incoming and outgoing traffic of the API provider. This positioning allows the attacker to observe both the originating IP addresses of users connecting to the API provider and the ADNL traffic between the provider and liteservers.

By applying our keystream recovery attack, the adversary can decrypt ADNL traffic between the API provider and liteservers, revealing information such as wallet addresses and other blockchain activities. The adversary can then correlate this traffic with the incoming user connections through various metadata such as timing patterns and packet sizes, effectively linking users' network identities to their blockchain activities. This linking undermines one of the fundamental privacy properties users expect from cryptocurrency systems.

6. Breaking ADNL's Integrity

The key extraction attack detailed in the previous section enables an attack on ADNL's integrity mechanism. While the protocol attempts to provide integrity through a hash-based construction, knowledge of the encryption keystream undermines this protection. While blockchain applications can implement additional integrity checks such as block validation through Merkle proofs, these verifications often incur significant computational overhead. Consequently, certain operations may rely primarily on transport-level integrity. Here, we analyze how the integrity mechanism can be broken and examine the practical challenges of exploiting this vulnerability in real-time scenarios.

Our attack implementation detailed in 7 demonstrated this vulnerability by targeting account balance queries. When an end-user wallet requests an account balance, the request flows through an API provider to a liteserver. Both the API provider and the wallet accept the balance response from the liteserver without additional cryptographic verification. By compromising the transport integrity, we successfully modified these balance responses, causing both the API provider and the end wallet to display incorrect balance information.

6.1. Compromising the Integrity Mechanism

ADNL's integrity mechanism relies on the secrecy of the nonce, which as shown in Table 1 is a 32-byte random value included in each datagram. Each datagram contains a hash computed as $\mathrm{SHA\text{-}256}(\texttt{nonce}\|\texttt{buffer})$, where the nonce is treated as a secret value known only to the communicating parties. However, once an attacker recovers the encryption keystream through the two-time pad attack, they can decrypt the original datagram to obtain both the nonce and the original data, modify the data arbitrarily and compute $\mathrm{SHA\text{-}256}(\texttt{nonce}\|\texttt{modified\_data})$, then re-encrypt both the modified data and new hash using the extracted keystream. The result is a valid encrypted datagram which passes the ADNL integrity check.

6.2. Real-time Attack Considerations

The main challenge of the active attack is maintaining low latency. To remain covert, the attack must not introduce delays that would make the network behavior appear suspicious. Our attack necessarily introduces latency as server responses must be delayed to allow time for keystream recovery. During this delay, we perform several operations: capturing the traffic, executing replay operations, and running the keystream extraction algorithm. These operations must keep pace with the server-to-client keystream ($k_{s\to c}$), which progresses with each encrypted response. The need to minimize latency motivated our optimization efforts in the keystream recovery algorithm, detailed in Section 5.2.3. In Section 7.2.4, we demonstrate that the throughput of our key recovery algorithm can keep pace with actual server responses, and when run continuously, achieves our latency target of sub-second delays.

6.3. Attack Implications: Financial Deception

By exploiting the broken integrity mechanism, an attacker can manipulate financial information displayed to users, potentially enabling double-spend like attacks without modifying the actual blockchain state. The attacker could implement a *balance spoofing attack* by intercepting balance queries and modifying responses to display inflated balances. This could deceive users into believing they've received funds that were never actually sent. For example, in a transaction where the victim agrees to provide goods or services after receiving payment, the attacker could make it appear that the payment has arrived when it has not. Another concerning scenario is *asset price manipulation*, where the attacker modifies responses containing token price information. By artificially inflating the price of worthless tokens, attackers could trick users into accepting these tokens as payment. For instance, an attacker might send a victim tokens with no real market value while making them appear worth thousands of dollars through manipulated API responses.

7. Attack Implementation

Our end-to-end experimental setup consists of three primary components: (i) an instance of the opentonapi client (v1.2.2) under our control, (ii) an actual TON liteserver from the mainnet network, and (iii) our attack infrastructure. The opentonapi client connects to the liteserver using the ADNL protocol over TCP. Our attack implementation establishes a man-in-the-middle position between these components, allowing us to intercept and analyze all ADNL traffic. Specifically, we capture the initial ADNL handshake packet, intercept all encrypted requests from the client to the server, perform multiple replays of these requests to the server, and

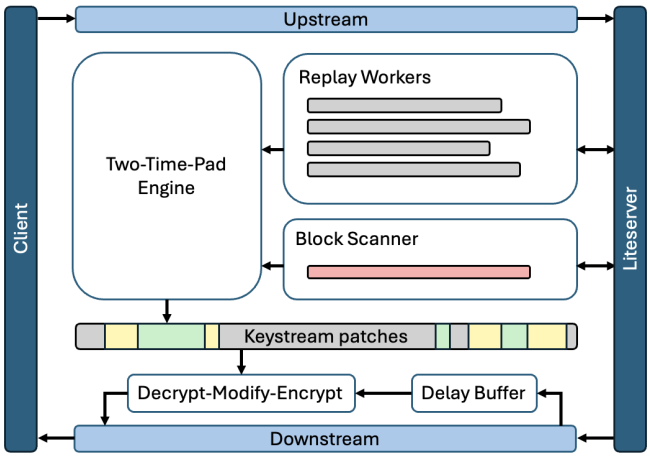

Figure 2. Attack architecture showing the main components: replay workers and blocks collector feed intercepted ciphertexts and predicted plaintexts to the Two-Time-Pad engine, which recovers keystream patches used by the response modifier to modify intercepted liteserver responses.

analyze the resulting encrypted responses. To evaluate the response tampering attack, we also connected a tonkeeper wallet [35] to our controlled opentonapi instance, which allowed us to demonstrate the attack by modifying the wallet balance information displayed to the end-user. All captured sessions were exclusively between our controlled components, ensuring no impact on other users' traffic or the liteserver's normal operation. This setup allowed us to perform a complete proof-of-concept of both the confidentiality and integrity attacks described in this paper. This section details the system architecture of our attack and provides an evaluation of its effectiveness in a real-world setting. An implementation of our attack, which can reproduce our results, is publicly available [14].

7.1. System Architecture

Our attack system consists of five modules that work in concert to intercept, analyze, decrypt and modify ADNL communications. Each module is designed to handle a specific aspect of the attack.

Network Interceptor. The Network Interceptor module operates at the TCP layer between the client and the liteservers it connects to. In our setting, the client is the opentonapi service. While this opentonapi service maintains connections to multiple liteservers, it prioritizes a "best connection" which carries the majority of the traffic. This high-traffic ADNL connection, carrying both indexer and other potentially interesting requests, becomes our primary target. The Network Interceptor performs the following operations:

- Monitors TCP sessions between client and liteservers
- Identifies and focuses on the "best connection"
- Captures the entire request stream, including the handshake phase. In scenarios where a complete capture isn't available, injects TCP RST packets to force connection resets
- Captures the response stream
- Tags captured traffic with timestamps for correlation with the Blocks Collector responses
- Controls the timing of response delivery to the client to provide sufficient time for the analysis
- Modifies server-to-client traffic based on interrupts from the Response Modifier

Replay Workers. The Replay Workers module manages the controlled replay of captured ADNL sessions. The module executes exact replays of the request stream captured by the Network Interceptor, and uses buffering strategy to exploit the request multiplexing property of liteservers described in Section 4.3. This buffering strategy designed to maximize response reordering:

- Divides the captured request stream into segments
- Transmits each segment in rapid succession (burst)
- Varies segment boundaries across different replays to enhance reordering entropy
- Maintains different timing patterns between replays while preserving ordering within bursts

Blocks Collector. The Blocks Collector module gathers block data independently through legitimate liteserver connections. It implements an adaptive collection strategy based on timestamps from the Network Interceptor and feedback from the Two-Time-Pad Engine regarding successful matches. This module makes `getBlock` requests to gather block data. Each `getBlock` request takes a Unix timestamp (in seconds) as input. The module employs an iterative collection strategy for scenarios involving large time frames:

- Initial sparse sampling: Requests blocks at fixed intervals (e.g., every 10 seconds) around the target timeframe we get from the Network Interceptor
- Upon receiving positive match feedback from the Two-Time-Pad Engine, narrows its focus to the time window around successful matches
- Progressively increases sampling density in promising time regions until complete coverage is achieved

Two-Time-Pad Engine. The Two-Time-Pad Engine implements the keystream recovery algorithm detailed in Algorithm 3. It processes inputs from both the Replay Workers and Blocks Collector to recover the server-to-client keystream $k_{s \to c}$. The Two-Time-Pad engine receives:

- A set of ciphertext streams $c_1, ..., c_N$ from the N Replay Workers
- A set of predicted packets $\hat{P}$ from the Blocks Collector

As matches are identified between the predicted packets and the intercepted ciphertext streams, the engine emits feedback to the Blocks Collector, enabling it to refine its block collection strategy. This feedback mechanism creates an adaptive loop: successful matches guide the Blocks Collector toward

promising time windows, which in turn provides more relevant predicted packets for keystream recovery.

Response Modifier. The Response Modifier module operates on decrypted server-to-client traffic using keystream data recovered by the Two-Time-Pad Engine. It performs the following operations:

- Receives server-to-client traffic captured by the Network Interceptor
- Decrypts responses using the recovered keystream $k_{s \to c}$
- Scans decrypted traffic for targeted patterns
- Modifies identified packets according to attack requirements
- Re-encrypts modified packets
- Issues modification instructions to the Network Interceptor

7.2. Evaluation

In this section we provide a detailed evaluation of our attack's performance across multiple dimensions. As these evaluations demonstrate, our attack can be effectively executed using consumer-grade hardware, with computational requirements modest enough to be within the capabilities of motivated attackers targeting high-value cryptocurrency transactions.

7.2.1. Evaluating Request Multiplexing. To analyze how request timing affects response ordering, we measure the derangement ratio $D(\sigma)$ across different request transmission patterns. Recall that $D(\sigma) = \frac{|j:\sigma(j)\neq j|}{n}$ represents the fraction of responses appearing at positions different from their corresponding requests, with values ranging from 0 (no reordering) to 1 (maximum reordering).

We conducted experiments transmitting sequences of 100 requests while varying the interval between consecutive transmissions. Figure 3 plots the relationship between request transmission intervals and the resulting derangement ratio. When requests are sent with intervals exceeding 100ms, responses largely maintain the original request order, yielding low $D(\sigma)$ values. However, transmitting requests in rapid succession with intervals under 10ms consistently produces high derangement ratios approaching 1.0. This analysis demonstrates that controlled request timing can reliably induce response reordering. By transmitting requests in rapid bursts, we can ensure that matching response packets appear at different positions across replayed sessions.

7.2.2. Segment Matching Benchmarks. To evaluate the computational performance of our optimizations, we conducted benchmarks measuring batched segment matching between $|N|$ streams of 5MB each and $|\hat{P}|$ streams varying in size from 32KB to 100KB. We conducted benchmarks comparing our optimized implementation against a baseline that lacks the key optimizations described in Section 5.2.3. The baseline implementation computes cross-correlation directly using NumPy's correlate function, performing $|N| \cdot |\hat{P}|$ separate correlations, and processes the full byte stream without splitting it into binary channels.

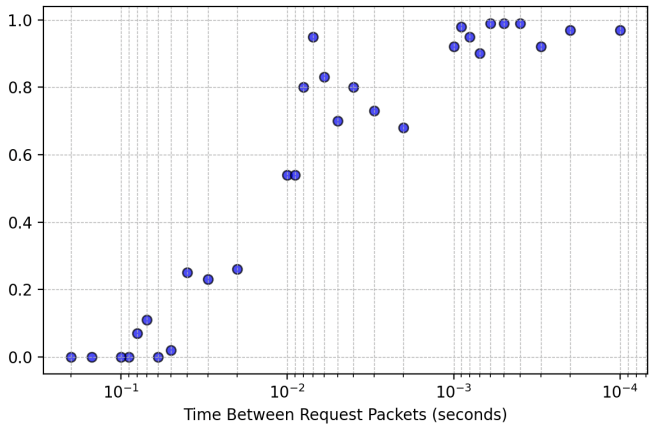

Figure 3. Derangement ratio measurements as a function of interval between consecutive request packets, showing that intervals under 10ms reliably induce response reordering with ratios approaching 1.0.

All benchmarks were performed on an Apple M1 Pro processor running at 3.2 GHz with 32 GB of RAM. All measurements were conducted using a single thread. Table 2 presents the CPU time and memory consumption for various combinations of $|N|$ and $|\hat{P}|$.

TABLE 2. SEGMENT MATCHING BENCHMARKS

N	4	4	8	8
$\lvert\hat{P}\rvert$	10	20	10	20
Baseline Algorithm CPU (s)	155.5	313.5	318.7	647.7
Optimized Algorithm CPU (s)	9.54	17.76	18.63	34.41
Baseline Algorithm Mem (GB)	4.67	5.11	4.95	5.16
Optimized Algorithm Mem (GB)	2.39	3.19	3.37	3.69

7.2.3. Keystream Recovery Performance. We evaluated the performance of our keystream recovery algorithm using 5MB of liteserver response data generated in our attack setting described in Section 4.2, comprising 380 ADNL packets. The evaluation examined the algorithm's effectiveness across different numbers of replay streams (N), percentages of known plaintexts (L - as defined in Section 5.1.1), and different derangement ratio values $D(\sigma)$.

Figure 4 shows the keystream recovery rate with $D(\sigma) = 0.1$. With $N = 6$ and starting with 20% of known plaintext, the algorithm recovers approximately 47% of the keystream. Figure 5 shows the results given $D(\sigma) = 0.5$, indicating higher response re-ordering. With $N = 6$ and 20% of known plaintext, the algorithm recovers more than 95% of the keystream.

These results validate our theoretical analysis: both higher derangement ratio values and more replay streams facilitate better keystream recovery by providing more opportunities for matching plaintext segments to appear at different offsets.

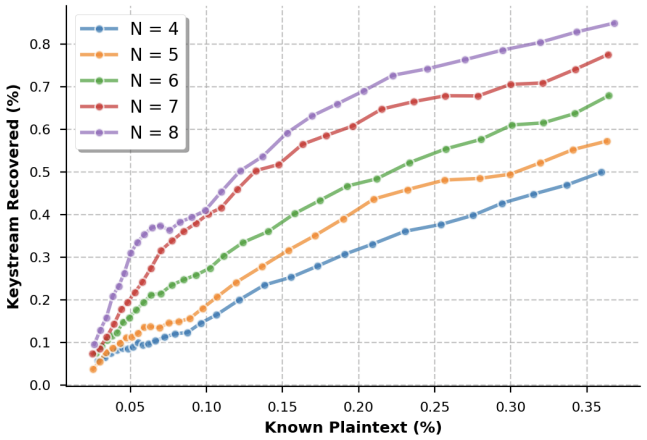

Figure 4. Keystream recovery performance with $D(\sigma) = 0.1$ for varying numbers of replay streams (N), showing increased recovery rates as N increases.

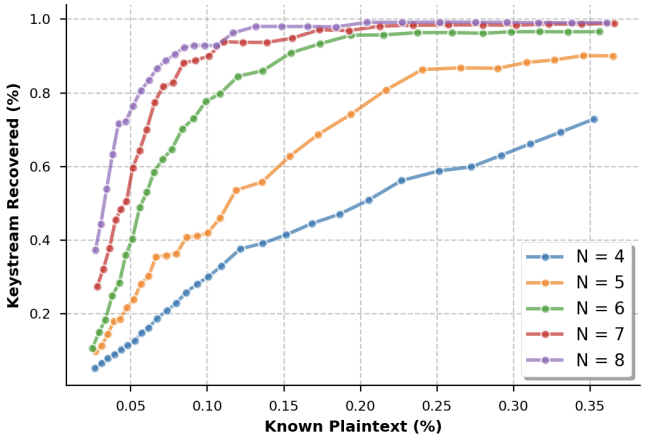

Figure 5. Keystream recovery performance with $D(\sigma) = 0.5$ for varying numbers of replay streams (N), demonstrating significantly higher recovery rates compared to $D(\sigma) = 0.1$ due to increased response reordering.

7.2.4. Active Attack Evaluation. We used the attack setting described at the beginning of this section to test the real-time performance of our active attack. Over a 37-second capture period, we collected 5 MB of response data across 380 packets, representing an average throughput of 135 KB/s. Of this traffic, 3.88 MB consisted of `getBlock` responses distributed across 41 packets, averaging 105 KB/s. Our attack configuration used 6 replay streams ($N = 6$). The blocks collector component gathered 10 known plaintext segments ($|\hat{P}| = 10$) totaling 1 MB, representing approximately 20% of the captured traffic. The keystream recovery algorithm, running with 4 threads and performing 5 iterations of batch segment matching, recovered 73.6% of the keystream in 35 seconds. This recovery rate demonstrates the practical feasibility of our attack. Higher workloads can be matched by running the keystream recovery algorithm using more threads.

7.2.5. Network Capacity Requirements. To provide concrete numbers for the network requirements of our attack, we fix the following parameters: (i) we assume the attacker is able of achieving 10% of known plaintext from the Blocks Collector, and (ii) we set a target of 80% keystream recovery.

These parameters were selected based on our successful instances of the active attack, with more stringent thresholds deliberately chosen to ensure confidence in the results. A key insight behind these constraints is that the 10% known plaintext threshold only needs to be achieved once during the initial bootstrapping phase, representing several MB of captured traffic. Once the self-sustaining loop is established through the 80% recovery rate, newly recovered plaintexts feed back into the system as known plaintexts, creating a reinforcing cycle that maintains attack effectiveness over time without requiring additional external plaintext inputs from the Blocks Collector.

With these constraints fixed, we analyzed different varying numbers of replay streams and different derangement ratio, from which we derived the network requirements to achieve the attack are minimum latency of 31ms, and at least seven replay streams. In our test environment, these numbers represent a peak bandwidth of 420KB/s. The minimum latency requirement is mainly a result of bursting requests to trigger the request multiplexing in liteservers, and represents an empirically observed threshold. The peak bandwidth varies based on application traffic characteristics, but in general, we can say that the system needs to handle about seven times the actual communication traffic, as indicated by the seven replay streams requirement.

8. Discussion

This paper presents an attack against the TCP variant of TON's ADNL protocol that enables both passive decryption of intercepted communications and active message modification. The attack leverages the protocol's semi-static DH key exchange, predictable server responses, and the server's message multiplexing behavior to recover plaintext from encrypted sessions, which can then be used to modify server responses. In this section, we discuss potential mitigation approaches, explore a direction for future work that could generalize our attack, and address the ethical considerations of our experiments.

8.1. Mitigation Approaches

From a security best practices perspective, transitioning to well-established, formally verified protocols like TLS 1.3 would provide the strongest protection against the class of vulnerabilities identified in this paper. Custom cryptographic protocols, even when designed by experienced developers, often fall victim to subtle vulnerabilities that established protocols have already addressed through years of cryptanalysis and standardization. We strongly advocate for the adoption of TLS 1.3 as the best practice for securing network communications in blockchain systems.

In response to our disclosure, the TON team implemented a fix in the ADNL client component of their core library. This solution adds a three-phase authentication protocol with messages that are transferred inside the already established ADNL channel:

1) The client sends a nonce
2) The server responds with its own nonce
3) The client completes the authentication by sending a fresh public key and a signature over both nonces

Beyond this implemented fix, we also observe that ADNL's UDP variant is already immune to the vulnerability presented in this paper due to its use of ephemeral DH keys in the server. By generating unique session keys across connections, the UDP variant prevents the replay attacks that enable our attack and also provides the additional benefit of forward secrecy against compromised server keys. This existing implementation demonstrates that the TON ecosystem already contains more secure design patterns that could be adopted more broadly. However, these observations should not detract from the primary recommendation: cryptographic protocols that handle sensitive user data should rely on established, well-tested solutions like TLS 1.3 rather than custom implementations.

8.2. Beyond Predicted Plaintexts

We propose a future research direction that could eliminate the need for predicted plaintext segments in our attack, potentially making it applicable to a broader range of scenarios where obtaining known plaintext is difficult.

The underlying two-time pad structure provides additional information we haven't yet exploited. The underlying plaintexts contain relationships in the form of identical segments appearing at different offsets. This setting shares some similarities with recent work by [22], where they analyze relationships between plaintexts that differ by small edits. Consider a packet p that appears at different positions across multiple streams. Let $i_1, ..., i_m$ denote the positions where p appears in streams $s_1, ..., s_m$ respectively. For any pair of occurrences (i_a, s_a) and (i_b, s_b), where $|p| = l$, we can write:

$$c_{s_a}[i_a : i_a + l] = p \oplus k[i_a : i_a + l]$$

$$c_{s_b}[i_b : i_b + l] = p \oplus k[i_b : i_b + l]$$

Even without knowing p, we can eliminate it by XORing these equations:

$$c_{s_a}[i_a : i_a + l] \oplus c_{s_b}[i_b : i_b + l] = k[i_a : i_a + l] \oplus k[i_b : i_b + l]$$

This equation relates different segments of the keystream. With sufficient such relations, we can construct a system of equations that constrains the possible values of the keystream. This observation suggests a potential generalization of our attack that doesn't rely on plaintext predictions. The primary challenge becomes identifying the positions where the same packet appears across different streams. One approach to address this challenge is to increase the number of replay streams beyond the minimum required for our current attack. With more replays, certain response packets are more likely to appear at the same position in multiple streams, creating patterns in the XORed ciphertexts that can be detected statistically. Such an approach would be particularly valuable in scenarios where predicting response content is infeasible.

8.3. Research Ethics Considerations

In conducting our experiments, we established a controlled test environment consisting of our own instance of the opentonapi ADNL client connecting to a mainnet liteserver. All sessions that were recorded or replayed as part of our experiments were exclusively our own test sessions. The experiments did not involve any sessions of other users, nor did they impact the normal operation of the servers or compromise any user data beyond our own. In retrospect, we acknowledge that these experiments could have been conducted on the testnet rather than the mainnet to further minimize any potential risks. For future research on blockchain protocols, we recommend using testnets whenever possible to avoid any ethical concerns while still achieving valid research results.

Acknowledgments

We would like to thank Daniel Katzan for introducing us to the ADNL protocol. We are grateful to Ben Riva for providing helpful comments that improved this work. We thank the TON Core team for their responsible handling of the issue and awarding us a bug bounty. Finally, we thank the IEEE S&P reviewers for their many helpful comments that substantially improved the quality of this paper.

References

[1] CoinMarketCap. Cryptocurrency prices, charts and market capitalizations. [Online]. Available: https://coinmarketcap.com/

[2] TON Nodes - Overview. TON Documentation. [Online]. Available: https://docs.ton.org/v3/guidelines/nodes/overview

[3] TON HTTP-based APIs. TON Documentation. [Online]. Available: https://docs.ton.org/v3/guidelines/dapps/apis-sdks/ton-http-apis

[4] ADNL over UDP Protocol. TON Documentation. [Online]. Available: https://docs.ton.org/v3/documentation/network/protocols/adnl/adnl-udp

[5] ADNL over TCP Protocol. TON Documentation. [Online]. Available: https://docs.ton.org/v3/documentation/network/protocols/adnl/adnl-tcp

[6] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/info/rfc8446

[7] E. Barker, L. Chen, A. Roginsky, A. Vassilev, and R. Davis, "Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography," National Institute of Standards and Technology, NIST Special Publication 800-56A Revision 3, 2018, accessed: 2024-11-13. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-56Ar3

[8] N. Borisov, I. Goldberg, and D. A. Wagner, "Intercepting mobile communications: the insecurity of 802.11," in *MOBICOM 2001, Proceedings of the seventh annual international conference on Mobile computing and networking, Rome, Italy, July 16-21, 2001*, C. Rose, Ed. ACM, 2001, pp. 180–189. [Online]. Available: https://doi.org/10.1145/381677.381695

[9] M. Vanhoef and F. Piessens, "Key reinstallation attacks: Forcing nonce reuse in WPA2," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, B. Thuraisingham, D. Evans, T. Malkin, and D. Xu, Eds. ACM, 2017, pp. 1313–1328. [Online]. Available: https://doi.org/10.1145/3133956.3134027

[10] B. Schneier and Mudge, "Cryptanalysis of microsoft's point-to-point tunneling protocol (PPTP)," in *CCS '98, Proceedings of the 5th ACM Conference on Computer and Communications Security, San Francisco, CA, USA, November 3-5, 1998*, L. Gong and M. K. Reiter, Eds. ACM, 1998, pp. 132–141. [Online]. Available: https://doi.org/10.1145/288090.288119

[11] H. Wu, "The misuse of RC4 in Microsoft Word and Excel," *IACR Cryptol. ePrint Arch.*, p. 7, 2005. [Online]. Available: http://eprint.iacr.org/2005/007

[12] J. Mason, K. Watkins, J. Eisner, and A. Stubblefield, "A natural language approach to automated cryptanalysis of two-time pads," in *Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS 2006, Alexandria, VA, USA, October 30 - November 3, 2006*, A. Juels, R. N. Wright, and S. D. C. di Vimercati, Eds. ACM, 2006, pp. 235–244. [Online]. Available: https://doi.org/10.1145/1180405.1180435

[13] W. T. Tutte, "FISH and I," Transcript of a lecture given on June 19, 1998, at the University of Waterloo, 1998.

[14] Anonymous. (2024) ADNL MitM Implementation. Implementation of Man-in-the-Middle attack against TON's ADNL protocol. [Online]. Available: https://anonymous.4open.science/r/adnl-mitm-E531

[15] TON Foundation. (2024) Fix handshake protocol security issue. TON Foundation. GitHub commit 8a41ee8. [Online]. Available: https://github.com/ton-blockchain/ton/commit/8a41ee8ffb7a22a3084a78d759ca66c8b25e2cc7

[16] C. M. Lonvick and T. Ylonen, "The Secure Shell (SSH) Protocol Architecture," RFC 4251, Jan. 2006. [Online]. Available: https://www.rfc-editor.org/info/rfc4251

[17] K. Seo and S. Kent, "Security Architecture for the Internet Protocol," RFC 4301, Dec. 2005. [Online]. Available: https://www.rfc-editor.org/info/rfc4301

[18] R. Canetti and H. Krawczyk, "Analysis of key-exchange protocols and their use for building secure channels," *IACR Cryptol. ePrint Arch.*, p. 40, 2001. [Online]. Available: http://eprint.iacr.org/2001/040

[19] T. Jager, F. Kohlar, S. Schäge, and J. Schwenk, "On the security of TLS-DHE in the standard model," in *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, ser. Lecture Notes in Computer Science, R. Safavi-Naini and R. Canetti, Eds., vol. 7417. Springer, 2012, pp. 273–293. [Online]. Available: https://doi.org/10.1007/978-3-642-32009-5_17

[20] H. Böck, A. Zauner, S. Devlin, J. Somorovsky, and P. Jovanovic, "Nonce-disrespecting adversaries: Practical forgery attacks on GCM in TLS," *IACR Cryptol. ePrint Arch.*, p. 475, 2016. [Online]. Available: http://eprint.iacr.org/2016/475

[21] National Security Agency, "The Venona Story," Center for Cryptologic History, Historical Publication. [Online]. Available: https://www.nsa.gov/portals/75/documents/about/cryptologic-heritage/historical-figures-publications/publications/coldwar/venona_story.pdf

[22] M. R. Albrecht, M. Backendal, D. Coppola, and K. G. Paterson, "Share with care: Breaking E2EE in nextcloud," *IACR Cryptol. ePrint Arch.*, p. 546, 2024. [Online]. Available: https://eprint.iacr.org/2024/546

[23] TON Foundation, "The Open Network Whitepaper," TON Foundation, Tech. Rep., July 2021, whitepaper. [Online]. Available: https://ton.org/whitepaper.pdf

[24] Tonstat. [Online]. Available: https://www.tonstat.com/

[25] Gamefi. [Online]. Available: https://ton.org/gamefi

[26] TON Network Configs. TON Documentation. [Online]. Available: https://docs.ton.org/v3/documentation/network/configs/network-configs

[27] TON Console API Documentation. TON Console. Accessed: 2024-11-13. [Online]. Available: https://docs.tonconsole.com/tonapi

[28] TON Center API v2 Documentation. TON Center. Accessed: 2024-11-13. [Online]. Available: https://toncenter.com/api/v2/

[29] APIs and SDKs - SDKs. TON Documentation. [Online]. Available: https://docs.ton.org/v3/guidelines/dapps/apis-sdks/sdk

[30] A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," RFC 7748, Jan. 2016. [Online]. Available: https://www.rfc-editor.org/info/rfc7748

[31] M. Dworkin, "Recommendation for Block Cipher Modes of Operation - Methods and Techniques," National Institute of Standards and Technology, NIST Special Publication 800-38A, 2001. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf

[32] H. Krawczyk, "The order of encryption and authentication for protecting communications (or: how secure is SSL?)," *IACR Cryptol. ePrint Arch.*, p. 45, 2001. [Online]. Available: http://eprint.iacr.org/2001/045

[33] R. Kuhn, K. Sriram, and D. Montgomery, "Border gateway protocol security," *NIST Special Publication*, vol. 800, p. 54, 2007.

[34] Opentonapi. Tonkeeper. Accessed: 2024-11-13. [Online]. Available: https://github.com/tonkeeper/opentonapi

[35] Tonkeeper Wallet. Tonkeeper. Accessed: 2024-11-13. [Online]. Available: https://tonkeeper.com/

[36] Proofs verifying (low-level). TON Documentation. [Online]. Available: https://docs.ton.org/v3/documentation/data-formats/tlb/proofs

[37] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, J. Dray, and S. Vo, "A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications," National Institute of Standards and Technology, NIST Special Publication 800-22 Revision 1a, 2010, accessed: 2024-11-13. [Online]. Available: https://csrc.nist.gov/publications/detail/sp/800-22/rev-1a/final

[38] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. Cambridge University Press, 2007, section on Convolution and Fourier Transforms. [Online]. Available: http://numerical.recipes/

[39] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson Education Limited, 2014.

[40] TL (Type-Language). TON Documentation. [Online]. Available: https://docs.ton.org/v3/documentation/data-formats/tl